

Rockchip Android14 Media Streaming AI字幕应用功能说明

文件标识: RK-SM-YF-F14

发布版本: V1.0.0

日期: 2025-08-04

文件密级: ☐绝密 ☐秘密 ☐内部资料 ☒公开

免责声明

本文档按“现状”提供, 瑞芯微电子股份有限公司 (“本公司”, 下同) 不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因, 本文档将可能在未经任何通知的情况下, 不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标, 归本公司所有。

本文档可能提及的其他所有注册商标或商标, 由其各自所有者所有。

版权所有 © 2025 瑞芯微电子股份有限公司

超越合理使用范畴, 非经本公司书面许可, 任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部, 并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址: 福建省福州市铜盘路软件园A区18号

网址: www.rock-chips.com

客户服务电话: +86-4007-700-590

客户服务传真: +86-591-83951833

客户服务邮箱: fae@rock-chips.com

前言

概述

本文档是基于RK3576 Android 14.0 Media Streaming的平台开发AI自动字幕的功能说明文档。

产品版本

芯片名称	Android版本
RK3576	Android14.0

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	yi.yang	2025-08-04	初始版本

目录

Rockchip Android14 Media Streaming AI字幕应用功能说明

1. 概述
2. 整体功能框架
3. 技术实现方案
 - 3.1 Framework层
 - 3.1.1 音频数据采集
 - 3.1.2 音频数据发送
 - 3.2 RkBoxAiService模块
 - 3.2.1 音频数据传输
 - 3.2.2 识别结果传输
 - 3.3 TvSettings模块
4. 功能使用方法
 - 4.1 开关设置
 - 4.2 效果演示
 - 4.2.1 开启：默认源 (`ai_lab_asr_en`)
 - 4.2.2 开启：中文源 (`ai_lab_asr_cn`)
 - 4.2.3 开启：会议模式 (`ai_lab_asr_meeting`)
5. FAQ
 - 5.1 单模块调试时，模块编译的输出路径和安卓设备的存放路径
 - 5.2 开启AI字幕功能后无字幕显示，如何排查

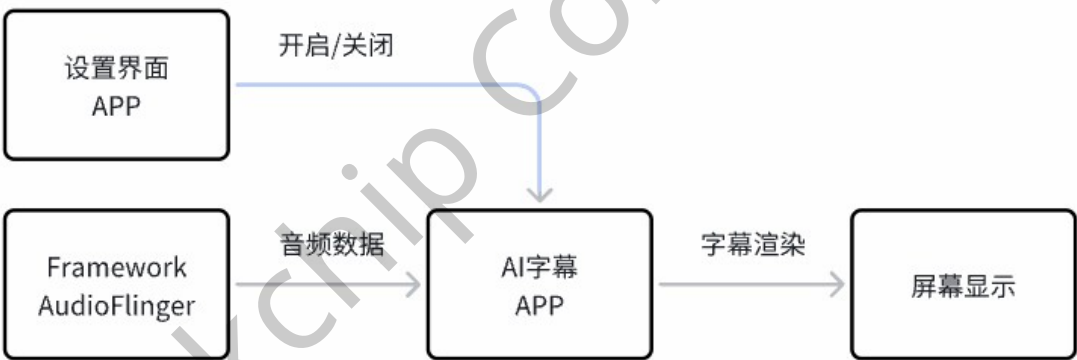
1. 概述

AI自动字幕是一个基于Android音频框架的实时语音转字幕系统，主要功能是通过音频输入（麦克风或系统音频）进行语音识别，并将识别结果以字幕形式展示在屏幕上，系统支持中英文双语识别，具备低延迟、高精度特性，特别针对视频会议场景设计了会议模式。

2. 整体功能框架

AI字幕功能按模块可以分为三大部分：

- 1. 音频数据获取（framework/av）：通过Framework层的 AudioFlinger 框架，获取音频输入设备（麦克风）或系统音频输出（如媒体播放音频流）的音频数据，使用 IRkAiCallback 接口发送音频数据至ASR模块APP。
- 2. 音频识别与字幕显示（RkBoxAiService）：即AI字幕的APP端，接收来自Framework层的音频数据，调用NPU的ROCKASR接口，将音频数据传入底层，获取来自底层的字符串识别结果，并渲染字幕显示。
- 3. UI设置界面（TvSettings）：用于在设置界面开启AI字幕功能，目前有默认源、中文源、会议模式三种模式。



具有以下功能特点：

- 1. 双通道语音识别
 - AT通路（系统音频）：捕获系统播放的音频（如视频声音、通话远端声音）
 - AR通路（麦克风音频）：捕获麦克风录音的音频
 - 会议模式：双通路独立处理，支持并行识别，双通道独立字幕显示
- 2. 实时字幕渲染
 - 悬浮窗显示字幕（支持原文/译文双行）
 - 超时管理：5秒无新语音自动隐藏字幕
- 3. 多语言支持
 - 中英文混合识别能力
 - 多种模式任意切换（通过系统属性控制）

3. 技术实现方案

3.1 Framework层

系统音频和录音音频的数据源都在Threads.cpp中获取并传到AI字幕APP端，所在线程如下表所示：

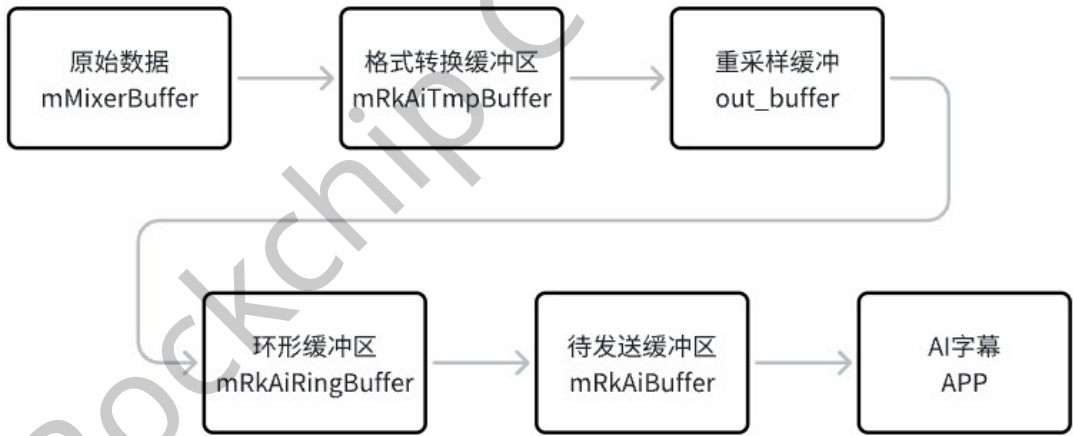
音频数据	处理线程
系统音频	PlaybackThread
录音音频	RecordThread

3.1.1 音频数据采集

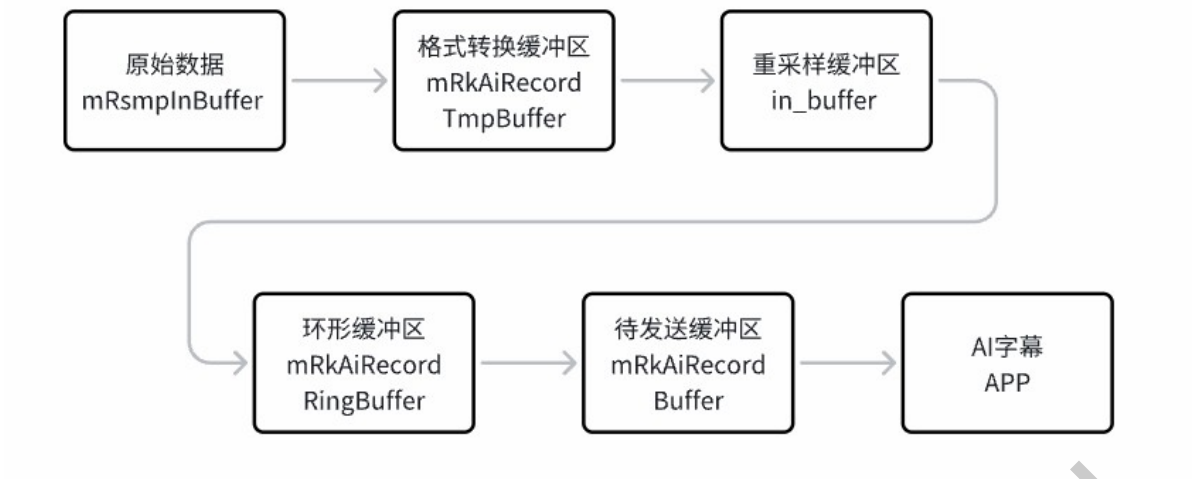
无论是系统音频还是录音音频，获取音频数据的流程都一样：

- 1.将原始数据转换为uint16_t的PCM音频格式
 - 2.重采样为底层ROCKASR要求的16KHz采样率
 - 3.传入RK定制的环形缓冲区，准备异步发送音频数据
 - 4.数据量满一帧后通过回调接口向AI字幕APP端发送一帧数据

PlaybackThread线程中的系统音频数据流如下图所示



RecordThread线程中的录音音频数据流如下图所示



3.1.2 音频数据发送

传到AI字幕APP的回调接口 `onAsrBuffer()` 有做修改，添加了参数 `int isAudioRecord`，用于判断系统/录音音频数据。

```
// media/libaudioclient/aidl/android/media/IRkAiCallback.aidl
/* Interface used to send pcm data from audioflinger to asrmanager */
interface IRkAiCallback {
    oneway void onAsrBuffer(in int[] buffer, int len, int isAudioRecord);
}
```

`int isAudioRecord` 对应的定义如下

```
#define ASR_AT_CHANNEL_INPUT 0 //系统音频数据
#define ASR_AR_CHANNEL_INPUT 1 //录音音频数据
```

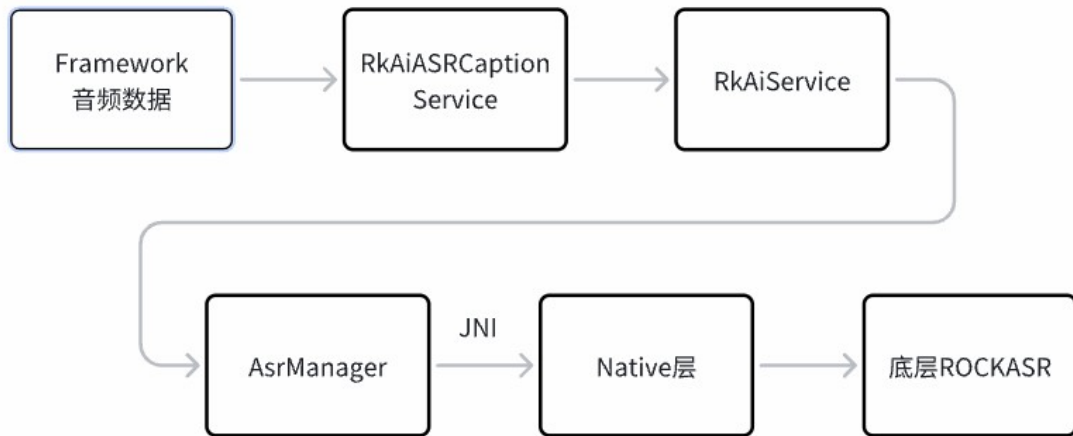
3.2 RkBoxAiService模块

即AI字幕的APP端，主要包含两个service

Service	作用
RkAiASRCaptionService	管理音频数据流 控制字幕显示控件
RkAiService	管理与Native层的通信

3.2.1 音频数据传输

获取的音频数据逐级传递到底层ROCKASR进行AI语音识别处理，音频数据流如下图所示



相关接口都使用 `int isAudioRecord` 判断系统/录音音频数据

```
// aidl/com/rockchip/aiservice/IRkAiService.aidl
boolean requestAsr(String packageName, in int[] inputBuff, int buffSize, int
isAudioRecord);
void flushAsrInput(String packageName, int isAudioRecord);

// src/com/rockchip/aiservice/util/JniTool.java
public native int native_asr_inference(short[] buffer, int readsize, int
isAudioRecord);
public native int native_asr_flush(int isAudioRecord);
```

```
// cpp/RkAiServiceJni.cpp
JNIEXPORT jint JNICALL native_asr_inference(
    JNIEnv *env, jobject thiz, jshortArray data, jint dataSize, jint
isAudioRecord)
```

3.2.2 识别结果传输

AI识别后ROCKASR返回字符串结果，直接通过回调的方式传递到RkAiASRCaptionService，最终显示在字幕中，字符串传递如下图所示



相关接口都使用 `ASR_CALL_STATE state` 判断返回字符串对应的识别结果

```
// src/com/rockchip/aiservice/util/JniTool.java
public interface MyAsrCallback {
    void asrInitCallBack(int ret);
    void asrCallBack(String result, ASR_CALL_STATE state);
}
```

```
// src/com/rockchip/aiservice/service/RkAiASRCaptionService.java
public void asrCallBack(String result, ASR_CALL_STATE state) {
    .....
}
```

ASR_CALL_STATE state 共有四个状态，定义如下

```
//cpp/RkAiServiceJni.h
#define AT_SOURCE 0 //系统音频原文结果
#define AT_RESULT 1 //系统音频翻译结果
#define AR_SOURCE 2 //录音音频原文结果
#define AR_RESULT 3 //录音音频翻译结果

//src/com/rockchip/aiservice/service/RkAiASRCaptionService.java
private final int AT_SOURCE = 0; //系统音频原文结果
private final int AT_RESULT = 1; //系统音频翻译结果
private final int AR_SOURCE = 2; //录音音频原文结果
private final int AR_RESULT = 3; //录音音频翻译结果
```

在RkAiASRCaptionService中通过 ASR_CALL_STATE state 判断当前返回的字符串应该显示在哪个Subtitle的TextView中，实际传递到 updateFloatingView() 中为 int whichSubtitle

```
private void updateFloatingView(String result, int whichSubtitle) {
    .....
    switch(whichSubtitle) {
        case AT_SOURCE : {
            .....
        } case AT_RESULT : {
            .....
        } case AR_SOURCE : {
            .....
        } case AR_RESULT : {
            .....
        } default :
            break;
    }
    .....
}
```

3.3 TvSettings模块

对于AI字幕应用，共有四个控制选项

```
<string-array name="ai_lab_asr_choose">
    <item>@string/ai_lab_asr_close</item>
    <item>@string/ai_lab_asr_en</item>
    <item>@string/ai_lab_asr_cn</item>
    <item>@string/ai_lab_asr_meeting</item>
</string-array>
```

英文对应字符如下


```
<string name="ai_lab_asr_close">close</string>
<string name="ai_lab_asr_en">default input</string>
<string name="ai_lab_asr_cn">chinese input</string>
<string name="ai_lab_asr_meeting">meeting mode</string>
```

简体中文对应字符如下

```
<string name="ai_lab_asr_close">关闭</string>
<string name="ai_lab_asr_en">开启：默认源</string>
<string name="ai_lab_asr_cn">开启：中文源</string>
<string name="ai_lab_asr_meeting">开启：会议模式</string>
```

在用户通过UI选定某个选项后，实际就是修改以下PROP属性然后重新启动AI字幕APP

```
private final String PROP_ASR_ENABLE_STATE = "sys.rkai.asr.enable";
private final String PROP_SUBTITLE_ENABLE_STATE = "sys.rkai.subtitle.enable";
private final String PROP_ASR_CHINESE_ENABLE_STATE = "sys.rkai.asr_ch.enable";
private final String PROP_ASR_MEETING_MODE_ENABLE_STATE =
    "sys.rkai.asr_meeting.enable";
```

其中 PROP_ASR_CHINESE_ENABLE_STATE 和 PROP_ASR_MEETING_MODE_ENABLE_STATE 是直接控制模式的属性，各属性含义如下：

序号	属性	含义
1	PROP_ASR_ENABLE_STATE (sys.rkai.asr.enable)	“0”： 关闭AI自动字幕功能 “1”： 开启AI自动字幕功能
2	PROP_SUBTITLE_ENABLE_STATE (sys.rkai.subtitle.enable)	目前作用同上，值保持同步
3	PROP_ASR_CHINESE_ENABLE_STATE (sys.rkai.asr_ch.enable)	“0”： 带翻译模式，支持英语->中文 “1”： 不翻译模式，支持英语/中文混合识别
4	PROP_ASR_MEETING_MODE_ENABLE_STATE (sys.rkai.asr_meeting.enable)	“0”： 关闭会议模式 “1”： 开启会议模式，目前绑定不翻译模式

对于各模式的取值如下：

模式（ 简体中文 ）	属性1	属性2	属性3	属性4
关闭	0	0	/	/
开启：默认源	1	1	0	0
开启：中文源	1	1	1	0
开启：会议模式	1	1	1	1

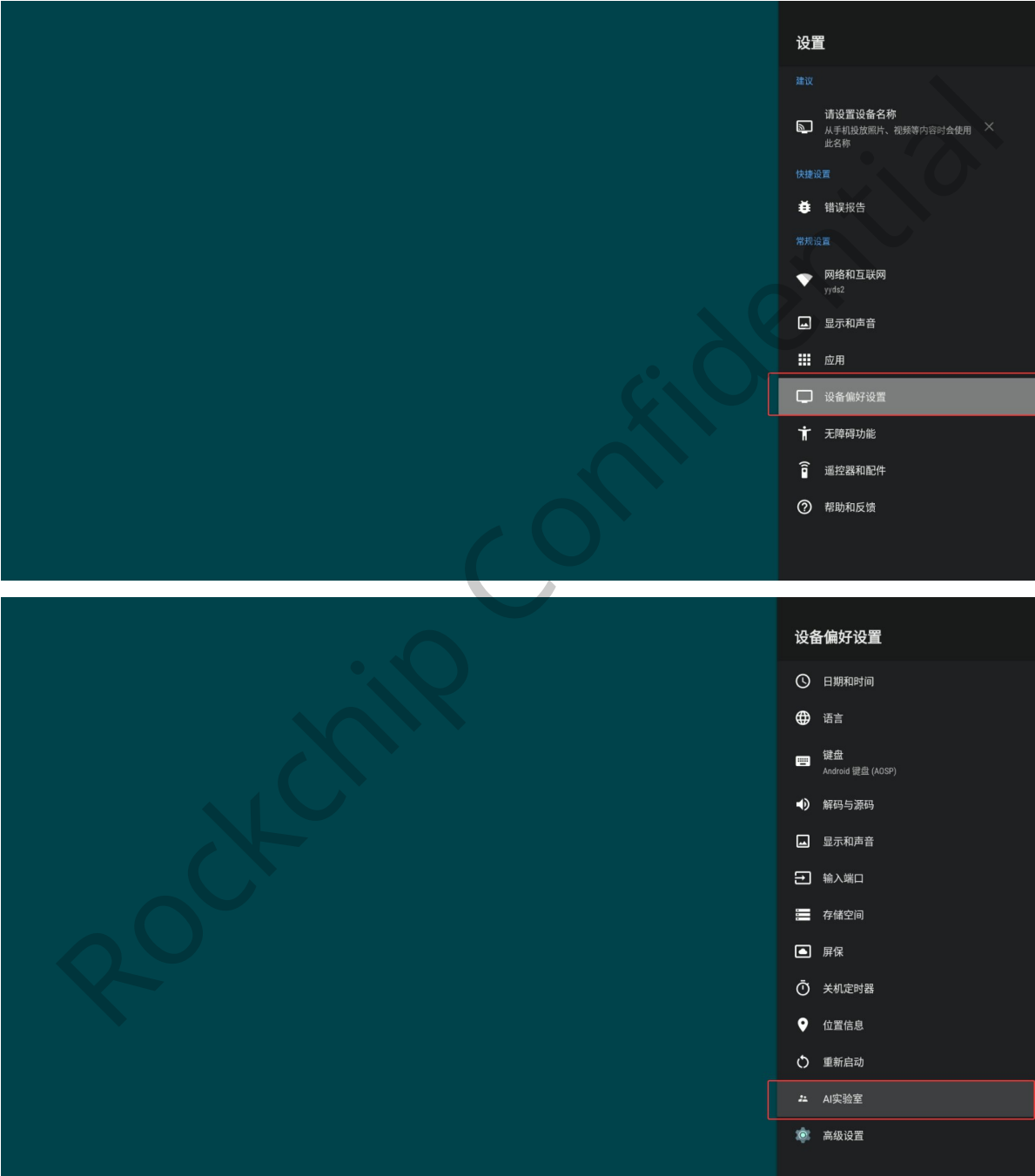
4. 功能使用方法

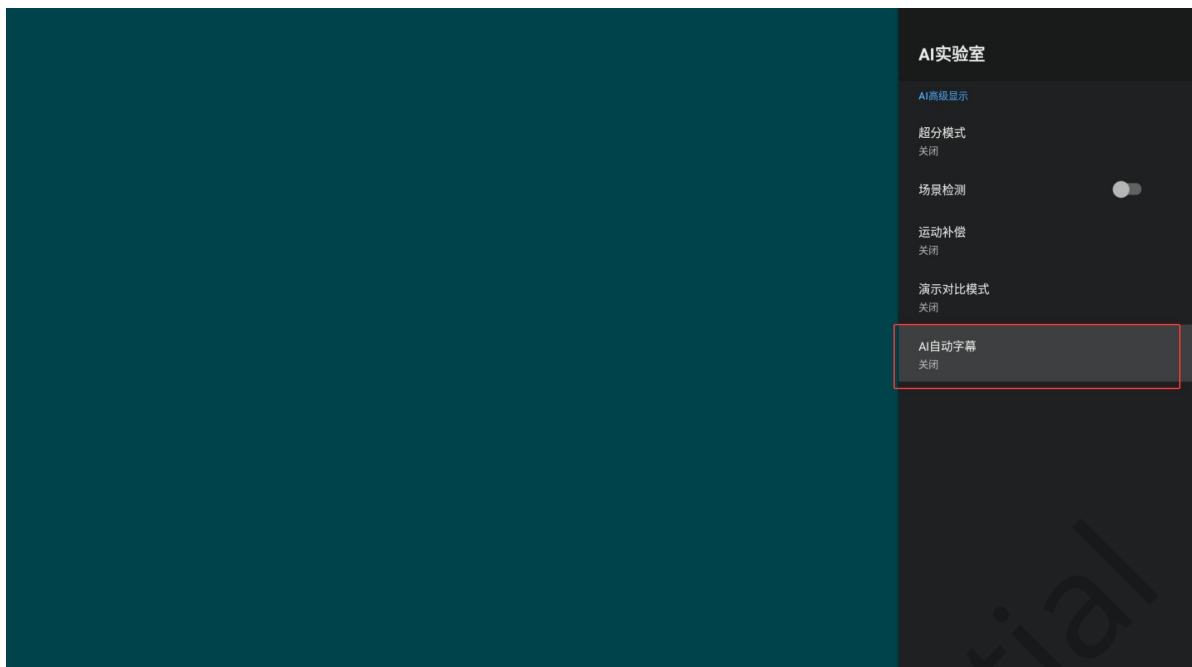
4.1 开关设置

AI自动字幕功能需在设置界面中手动开启，若系统的语言为简体中文，则跳转路径如下：

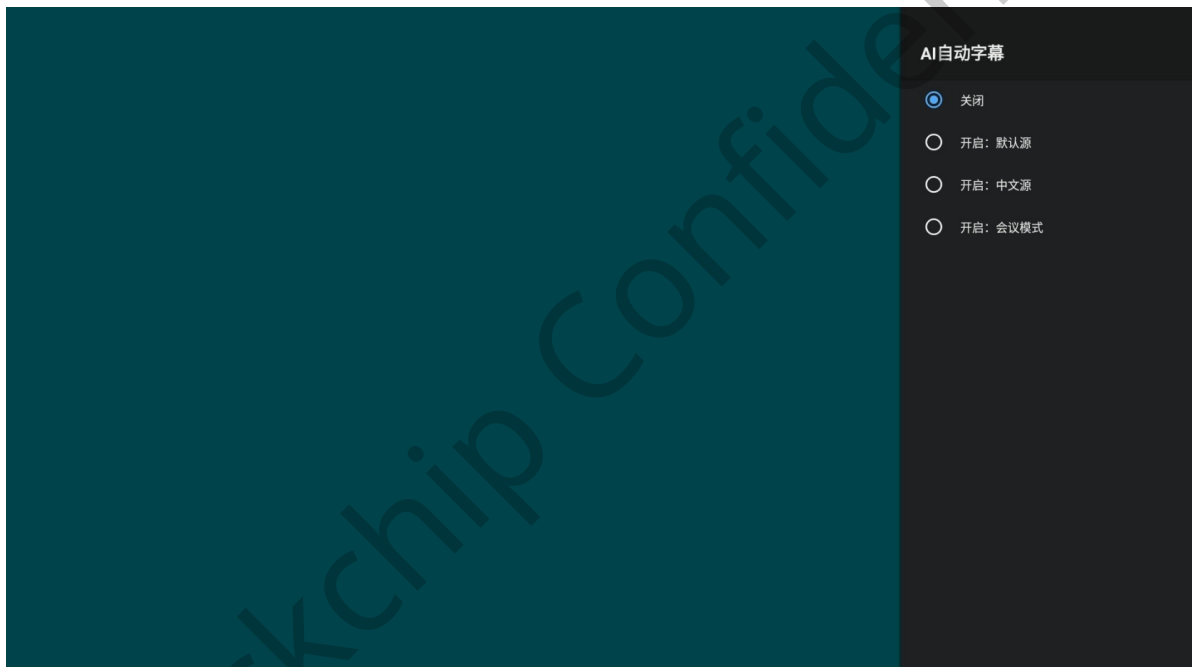
设置 -> 设备偏好设置 -> AI实验室 -> AI自动字幕

其他系统的语言同理





进入AI自动字幕设置界面，可以看到目前有四个选项，和3.3节的内容对应



4.2 效果演示

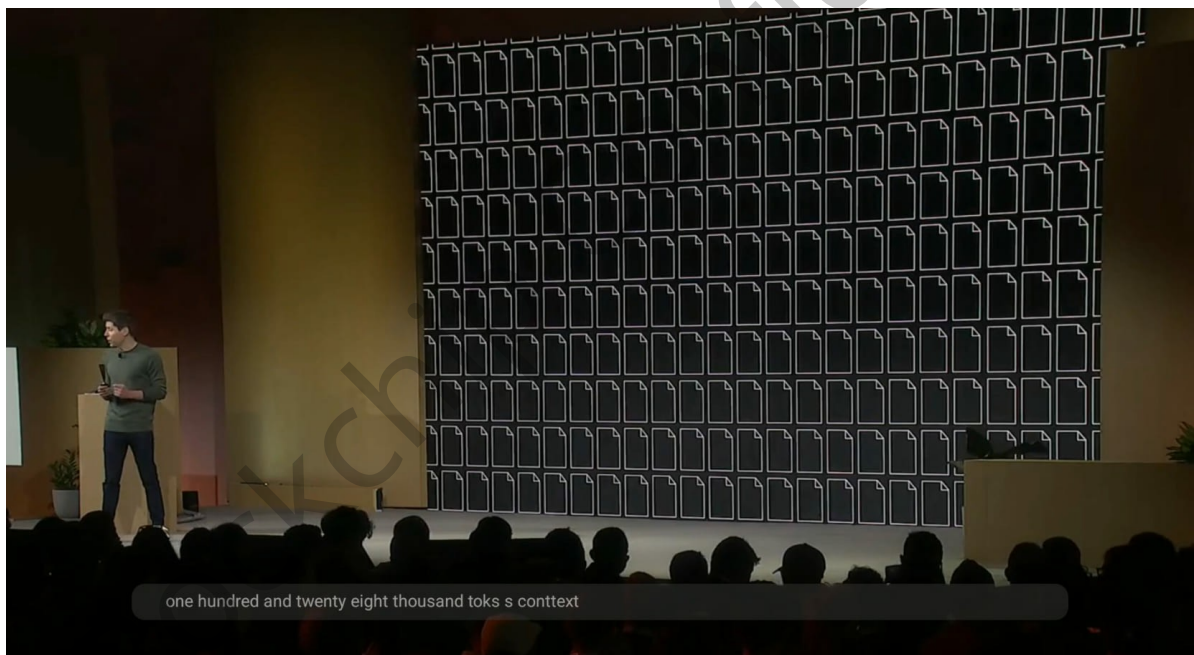
4.2.1 开启：默认源（`ai_lab_asr_en`）

该模式下仅捕获系统音频数据，并进行翻译，同时显示原文和翻译结果



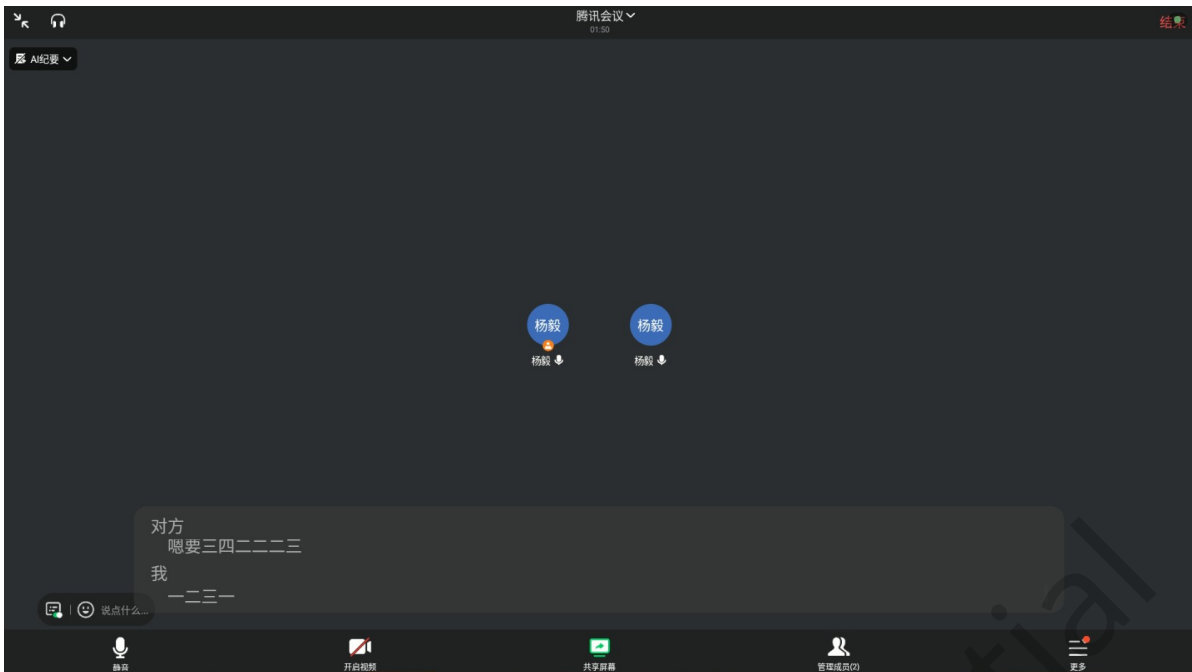
4.2.2 开启：中文源（`ai_lab_asr_cn`）

该模式下仅捕获系统音频数据，不进行翻译，仅显示原文识别结果，中文/英文双语识别



4.2.3 开启：会议模式（`ai_lab_asr_meeting`）

该模式下同时捕获系统音频和录音音频数据，不进行翻译，支持同时显示原文结果



5. FAQ

5.1 单模块调试时，模块编译的输出路径和安卓设备的存放路径

该功能涉及的模块较多，对应的路径都不一样，以下记录几个本功能的模块调试常用路径

模块	输出路径（AOSP目录）	下载路径（设备目录）
RkBoxAiService模块	out/target/product/rk3576_projector/system/app/RkBoxAiService/	system/app/
RkBoxAiService模块	out/target/product/rk3576_projector/system/lib64/librkai_service.so	system/lib64
TvSettings模块	out/target/product/rk3576_projector/system_ext/priv-app/TvSettings/	system_ext/priv-app

5.2 开启AI字幕功能后无字幕显示，如何排查

- 1.检查系统音量是否为0，静音情况下无法获取系统音频数据导致无字幕输出
- 2.检查是否有rkauth授权，adb查看 `/sdcard/rkai` 路径下是否有 `key_asr.lic` 文件

```
ls -l /sdcard/rkai
```

否则需要进行rkauth授权

```
//packages/apps/RkAuth/etc/rkauth_config.json
{
  "rkauth_modules_config": [
```

```

.....
{
    "module": "asr", //模块名: asr 语音转文字
    "activate_code_vendor_storage_id": 84, //asr在线密钥存储在vendorstorage
    的84位置, 这个id请勿修改。
    "activate_code_vendor_storage_size": 256,
    "license_vendor_storage_id": 1084,
    "license_vendor_storage_size": 1024,
    "activate_code_path": "",
    "license_path": "/sdcard/rkai/key_asr.lic" //RkAiService会读取这个文件
    进行asr的初始化
},
.....
]
}

```

3.检查设备是否有存放字幕功能相关的AI算法data文件, 若没有则需要重新导入

```
adb shell "ls /sdcard/rkai/asr_data"
```

4.检查编译镜像时device/rockchip/rk3576/rk3576_projector下的BoardConfig.mk是否有使能相关宏

```

#enable AIPQ
-BOARD_ENABLE_AIPQ_PROJECTOR_PRODUCT := false
+BOARD_ENABLE_AIPQ_PROJECTOR_PRODUCT := true

BOARD_SELINUX_ENFORCING := false
@@ -61,9 +61,11 @@ BOARD_HDR_SUPPORT := true
BOARD_SUPPORT_KEYSTONE := false

###support asr
-BOARD_SUPPORT_ASR := false
+BOARD_SUPPORT_ASR := true

```

5.尝试手动设置 `sys.rkai.subtitle.timeout`

```
adb shell "setprop sys.rkai.subtitle.timeout 5000" #单位ms, 字幕超时隐藏时间
```

若还是无效, 检查prop属性值是否符合3.3节所述

```
adb shell "getprop | grep sys.rkai"
```

6.若以上都未解决, 需要使用 `adb logcat` 抓取日志排查问题